

Tuya BeaconMesh SDK AK80x TuyaOS Guide

目录

1、SDK 概述.....	2
1.1 名词解释.....	3
1.2 环境搭建.....	3
1.3 SDK 架构.....	4
1.4 SDK 目录结构.....	5
2、快速体验.....	7
2.1 准备工作.....	7
2.2 创建产品.....	7
2.3 运行调试.....	11
3、开发步骤.....	12
4、应用开发指导.....	13
4.1 初始化.....	13
4.2 Loop.....	14
4.3 DP 下发数据处理.....	14
5、API 详细介绍.....	15
5.1 BeaconMesh 协议栈初始化.....	15
5.2 beacon dp 接收处理函数.....	16
5.3 beacon 发送函数.....	17
5.4 beacon loop 函数.....	18
5.5 beacon 启动配网函数.....	18
6、量产指导.....	19
6.1 购买授权码.....	19
6.2 芯片烧录授权.....	20
7、其它.....	21
7.1 涂鸦平台芯片烧录授权方式介绍.....	21
7.2 Beacon MESH 协议栈芯片移植指导.....	21
7.2.1 移植概述.....	21
8、FAQ.....	25

1、SDK 概述

可接入涂鸦平台的蓝牙协议包括：Tuya BLE 协议（简称：单点（单节点连接控制）、SIG Mesh 协议、BeaconMesh 协议。

涂鸦蓝牙协议分类	通信方式	典型特点	典型应用
Tuya BLE	与手机或网关建立 BLE 连接进行通信	直连、低功耗、传输速率快	手环、手表、智能牙刷、智能体脂秤
SIG Mesh	遵循 SIG 蓝牙 MESH 标准的蓝牙组网通信	网状网络、大规模群控	MESH 灯具、商照、智能家居套件
BeaconMesh	与手机或网关之间通过 BLE 广播进行通信	小数据包、近距离、超低速率、超低成本、支持转发	低成本门磁、PIR、温湿度传感器、低成本灯带、低成本墙面贴、遥控器、场景开关

其中 BeaconMesh 协议相对于 Tuya BLE 和 SIG Mesh 非常精简，其 SDK 特别适合直接移植到各种 BLE Beacon 芯片上，实现 SoC 级别的低成本应用。

BeaconMesh SDK 特点：

- 代码极简：总共约 1000 行 C 语言代码，可运行在 2K RAM + 16K ROM 的芯片上
- 跨平台：设计层面支持了跨硬件平台，只要实现 10 个以内的硬件和蓝牙广播相关函数，即可实现移植
- 功能完备：SDK 实现配对解绑、组包拆包、群组管理、自组网、转发、加密解密、安全策略等丰富的 IoT 通信能力

1.1 名词解释

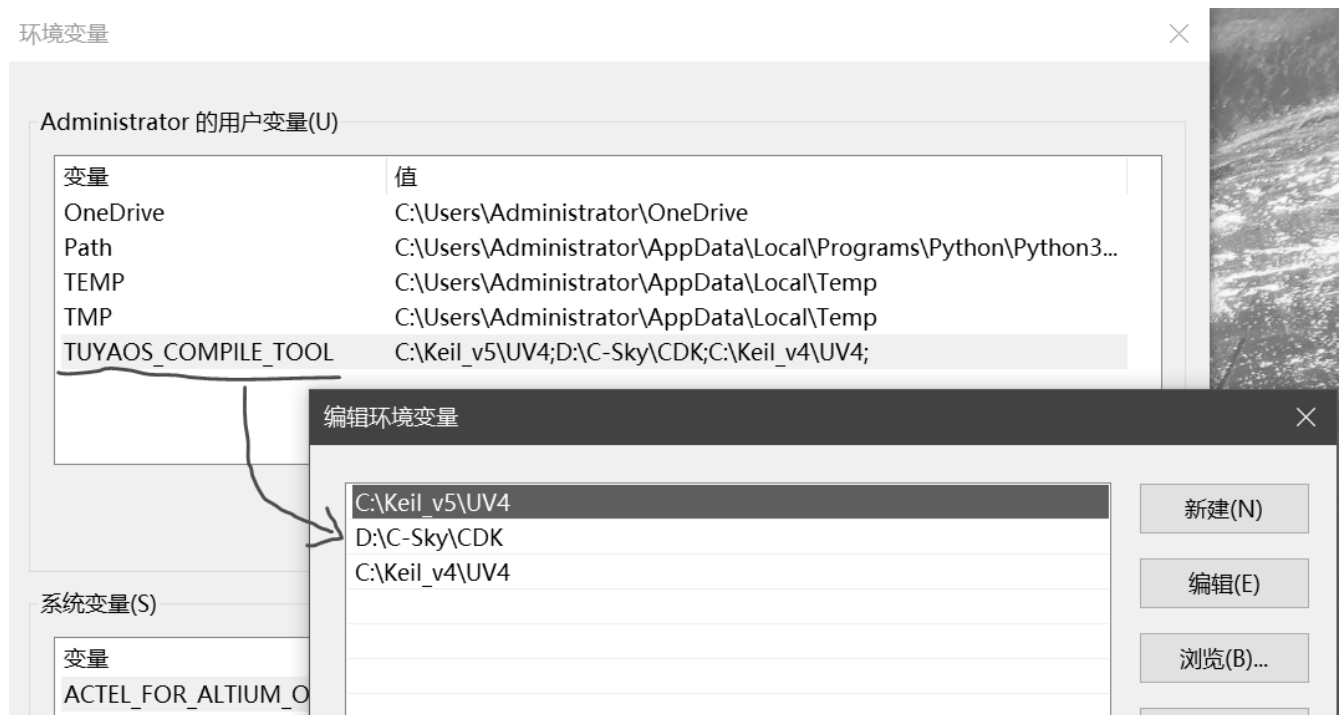
详细名词解释请参照：[涂鸦智能开发者名词解释](#)。

1.2 环境搭建

首先在 Windows 系统上搭建 Tuya Wind IDE 开发环境，参照：[Windows 系统](#)（只参考这一小节即可）

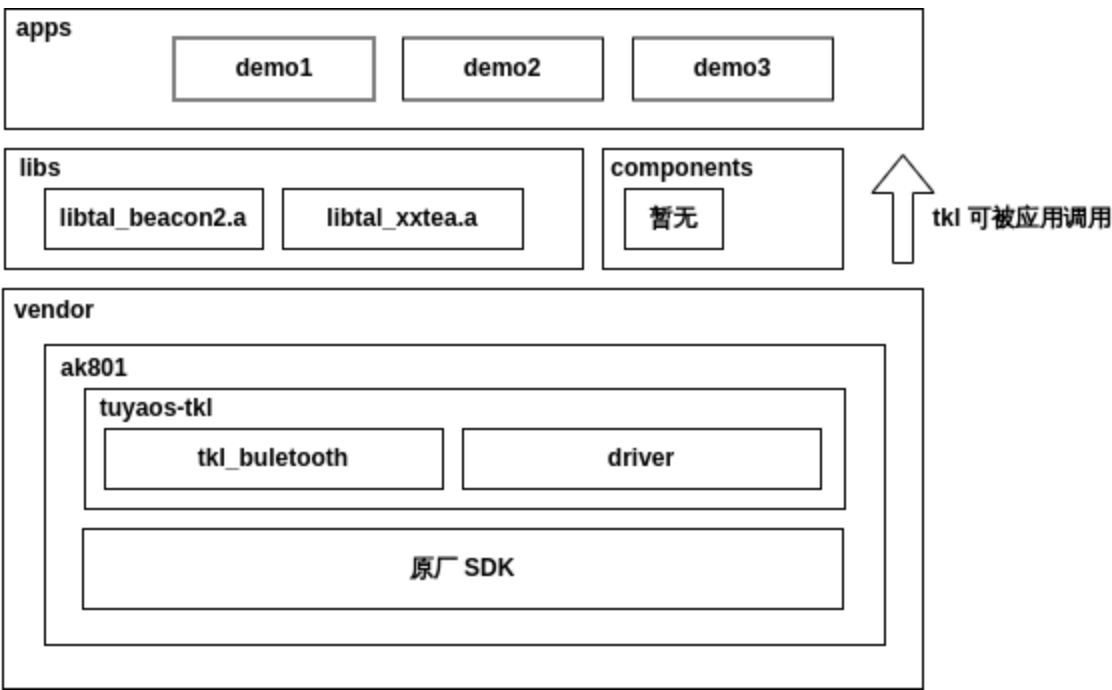
然后参考《SDK 开发包根目录/hardware/AK803AE-M3/chip_manual/开发环境搭建 V1.0.pdf》安装 CDK 开发环境（此时需要准备好 AK80x 开发板和 AK80x OTP 烧录器）

最后将 CDK 安装目录加入如下的环境变量中：



1.3 SDK 架构

SDK 架构图如下：

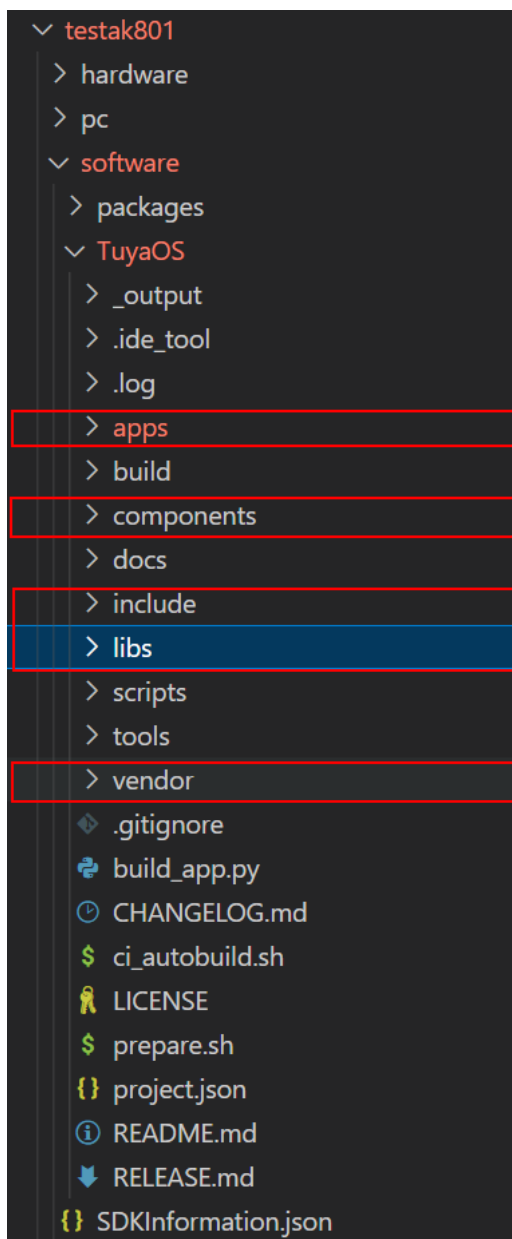


从底层向上：

- **vendor 层**：包含芯片原厂提供的 SDK，以及为了对接涂鸦体系的适配文件（蓝牙抽象层 `tkl_bluetooth`，硬件抽象层 `driver`（包含各种外设驱动））
- **libs 层**：
 - `libtal_beacon2.a` 包含涂鸦 beacon 核心协议栈
 - `libtal_xxtea.a` 包含加解密算法
- **components 层**：当前 demo 中暂时没有开源/应用组件
- **apps 层**：每个 demo 包含实现应用逻辑的全部文件

1.4 SDK 目录结构

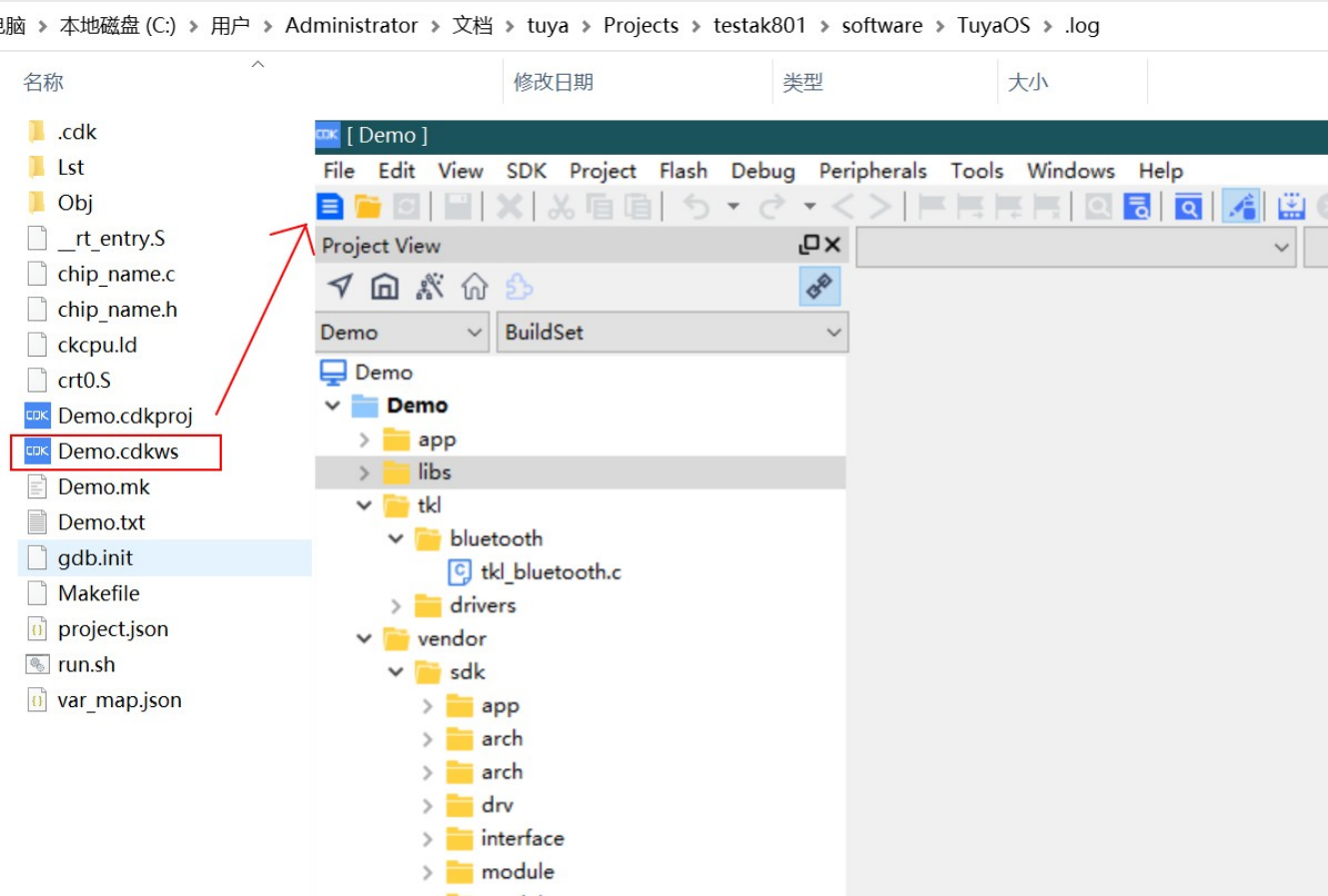
SDK 文件目录结构如下（在 VS-CODE 中打开）：



特别的：

- .ide_tool 是 TuyaOS 编译相关脚本（开发者不用关注）
- .log 是 TuyaOS 编译过程中的临时工程文件、临时编译文件
- _output 是 TuyaOS 产物生成文件夹

由于调试 AK801 需要借助专用的 IDE，因此这里介绍下如何找到用 Tuya Wind IDE 构建的 CDK 工程（图示是 AK801 专用 IDE 中打开目录结构）：



注虽能够用 CDK 对代码进行编译、调试、修改，但是务必注意：只修改现有工程中的现有文件的内容，不需要在 Tuya Wind IDE 中重新 build，但是一旦增删文件、修改配置，一定要重新在 Tuya Wind IDE 中 build。

因为这个 CDK 的工程是通过 build 临时自动生成的，单独在 CDK 中修改，在下一次重新构建时，会删除掉该临时工程文件。

2、快速体验

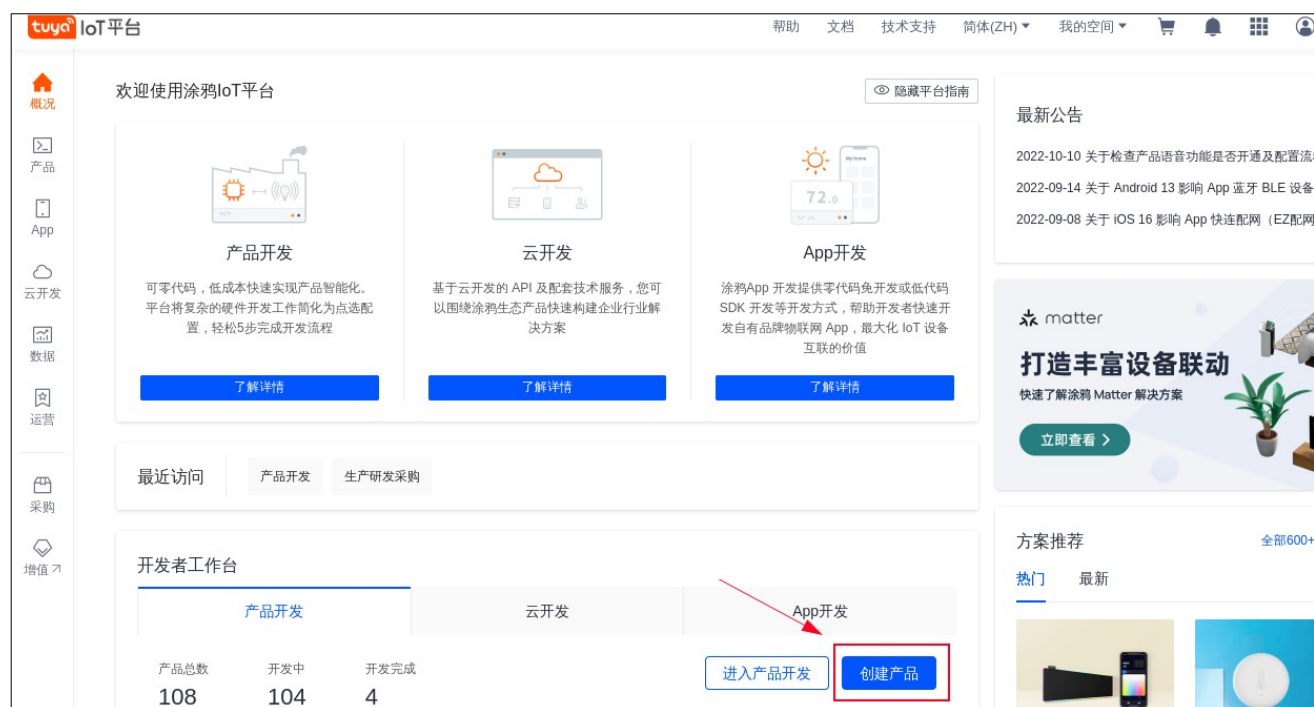
2.1 准备工作

- 1) 参考《1.2 环境搭建》将开发环境搭建好
- 2) 开发板购买链接：后续会上架，目前可以联系 BD 进行采购

2.2 创建产品

登录涂鸦开发者平台：<https://iot.tuya.com/>

STEP1：创建一个自定义开发的蓝牙 beacon 类型的灯带方案





采购

增值

数码娱乐

其他

4 完善产品信息

* 产品名称:

BeaconMesh

产品型号:

BeaconMesh

* 通讯协议:

☐ WiFi-蓝牙
 ☐ Wi-Fi
 ☐ 蓝牙Mesh(SIG)
 ☐ Zigbee
 ☐ 蓝牙
 ☐ 其他
 ☒ Tuya Beacon

STEP2：选择开关、模式

tuya IoT平台

帮助 文档 技术支持 简体(ZH) 我的空间

产品

添加标准功能

选择标准功能

其他可选功能

全部选择

亮度值

DP ID: 3 标识符: bright_value

>

冷暖值

DP ID: 4 标识符: temp_value

>

倒计时剩余时间

DP ID: 7 标识符: countdown

>

已选功能(2)

删除可选功能

开关

DP ID: 1 标识符: switch_led

×

模式

DP ID: 2 标识符: work_mode

×

STEP3：获取产品 ID (PID)

tuya IoT平台

帮助 文档 技术支持 简体(ZH) 我的空间

产品

BeaconMesh 切换产品

自定义开发

PID: bvlbaygh

品类: 灯带

通讯方式: Beacon

更多信息

编辑信息

01 功能定义

02 设备交互

03 硬件开发

04 产品配置

05 测试服务

指南&任务

标准功能、自定义功能（可选）、高级功能（可选）共同定义产品功能，设备控制面板和固件都与产品功能有关。

如何定义产品功能？

标准功能(个)

自定义功能(个)

高级功能(个)

2

0

1 / 3 查看

标准功能

DP ID	功能点名称	标识符	数据传输类型	数据类型	功能点属性	备注
1	开关	switch_led	只下发 (wr)	布尔型 (Bool)		编辑

STEP4：获取激活码



等待大约 1 ~ 2 分钟，授权码清单生成，点击下载即可：



清单格式如下：

uuid	key	mac	
tuyae15aa4acb29dc46d	RAjczGyAK4lvmPIIXGpKIKwHiyRayKsg	DC234D28EBEB	举例，不是正确的数据

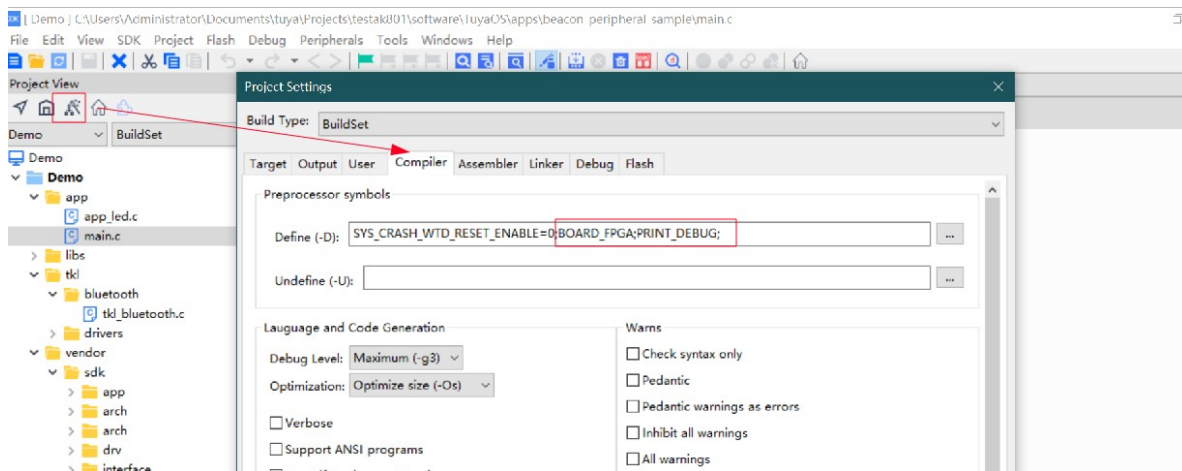
2.3 运行调试

1) 三色灯连接：

开发板 IO 丝印	代号	PIN
4	高点亮 LED（外部电路最好有下拉电阻）	
15	UART_TX	IO15/TXD (9600)

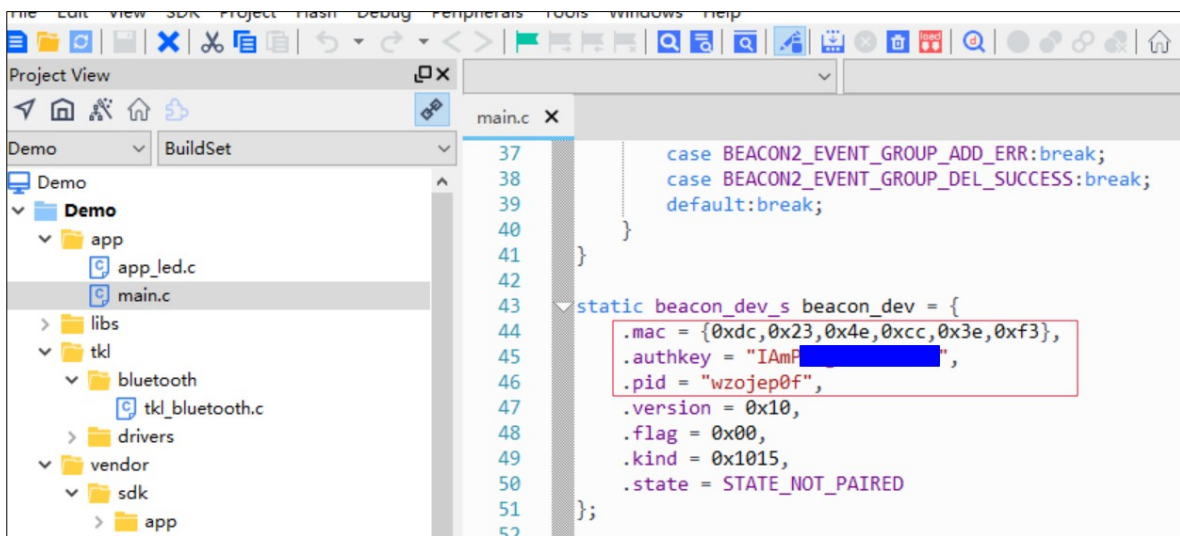
2) 三元组填写与开启调试模式

- 由于在开发板上看门狗、EEPROM 无法模拟，同时可以在调试状态时将 log 打开，这可以通过在工程配置页面新增两个宏实现：
 - BOARD_FPGA 使用开发板调试
 - PRINT_DEBUG 打开 9600 串口



注：实际生产时需要去掉 BOARD_FPGA 和 PRINT_DEBUG。

- 修改 main.c 中的 pid 为你自己创建产品的 ID，authkey 为购买激活码中的任意一个（只取前 16 字节），mac 地址一定要和所取 authkey 对应：



启动调试后，LED 灯闪烁，表示进入配网状态（60S 超时），此时使用涂鸦智能便可搜索、添加、控制设备，串口也有一些 log 帮助理解程序。

3、开发步骤

Tuya Wind IDE 开发环境搭建 -> 在 IoT 平台创建产品 -> 为产品设计合理的 DPs -> 为产品设计/选择合适的控制面板 -> 开发并测试 -> 发布产品进行小批量

注：

- 1) beacon 适合短数据，单包交互，不适合长数据，多包交互；
- 2) DP 有效数据最长支持 14 字节；

4、应用开发指导

以《app/beacon_peripheral_sample》DEMO 为例（每个 DEMO 的应用开发指导存放在其对应应用文件夹下）。

4.1 初始化

```
VOID_T tal_framework_init_first(VOID_T){
    ...
}

VOID_T tal_framework_init_second(VOID_T){
}

VOID_T tal_framework_init_third(VOID_T){
    tkl_ble_stack_init(0); // 蓝牙协议栈初始化
}

VOID_T tal_framework_init_last(VOID_T){
    app_led_init(); // led 初始化

    ty_beacon2_node_init(&beacon_dev); // BeaconMesh 协议栈初始化
    if(beacon_dev.state == STATE_NOT_PAIRRED){ // 如果设备没有配网
        ty_beacon2_node_pair_start(60000000); // 启动设备配网，设置 1min 超时
        app_led_blink(240); // 闪烁 led 1min
    } else { // 如果处于已经配网状态，上电后主动上报心跳
        u8 payload[16];
        memset(payload, 0, 16);
        payload[0] = 0x01;
        frame_send(0x08, 0x8000, 0x02, payload, beacon_dev.beaconkey, 0); // 上
        电上报心跳，不转发
    }
}
```

初始化函数主要实现：蓝牙协议栈初始化、BeaconMesh 协议栈初始化、根据 EEPROM 中记录的设备是否配网状态，决定上电后进行配网广播还是上报心跳。

4.2 Loop

```
VOID_T tal_framework_loop(VOID_T){
    ty_beacon2_node_run();//beacon mesh 节点主逻辑维护进程
    app_led_run();//led 闪烁维护进程
    app_led_reset_run(&beacon_dev);//三次快速上电重置逻辑维护进程
}
```

主循环逻辑中主要有三个非阻塞进程函数：beacon mesh 节点协议栈主逻辑进程、led 闪烁维护进程、三次快速上下电重置维护进程。

4.3 DP 下发数据处理

```
void app_dps_download(u8 dpid, u8 dpty, u8 dplen, u8 *dpvalue){
    PR_DEBUG("dpid=%d dpty=%d, dplen=%d, dpvalue=[", dpid, dpty, dplen);
    PR_DEBUG_HEX_ARRAY(dpvalue, dplen);
    PR_DEBUG_RAW("]\n");
    switch(dpid){
        case 0x01://onoff
            app_led_onoff(dpvalue[0]);
            break;
        case 0x02://mode:white,color,scene,music
            break;
        default:
            break;
    }
}
```

DP 下发处理函数，对应着平台上创建的 DPs，开发者自定义开发时，自己新增的 DPs 的处理可以放到这里。

5、API 详细介绍

5.1 BeaconMesh 协议栈初始化

函数	u8 ty_beacon2_node_init(beacon_dev_s *p_beacon_dev)	
作用	初始化 beacon mesh 协议栈	
入参	beacon_dev_s *p_beacon_dev	
	.mac	mac 6 bytes
	.authkey	授权码中 authkey
	.pid	产品 ID
	.version	版本, 0x13 → 1.3
	.kind	大小类 (同 SIG MESH)
返回		

示例:

往往在程序初始化的地方调用:

```
static beacon_dev_s beacon_dev = {
    .mac = {0xdc, 0x23, 0x4e, 0xcc, 0x3e, 0xf3},
    .authkey = "IAmPFxxxxxxxxxxxx",
    .pid = "wzojep0f",
    .version = 0x10,
    .flag = 0x00,
    .kind = 0x1015,
    .state = STATE_NOT_PAIRRED
};

ty_beacon2_node_init(&beacon_dev);
```

5.2 beacon dp 接收处理函数

函数	void app_dps_download(u8 dpid, u8 dpty, u8 dplen, u8 *dpvalue)	
作用	从网关或手机发来的 dp 数据回调函数	
入参	u8 dpid	DP ID
	8 dpty	DP 类型
	u8 dplen	DP 数据的长度
	u8 *dpvalue	DP 数据
返回		

示例：

往往在 main.c 中实现，用于处理 app 或网关发来的 dp 命令：

```
void app_dps_download(u8 dpid, u8 dpty, u8 dplen, u8 *dpvalue){
    PR_DEBUG("dpid=%d dpty=%d, dplen=%d, dpvalue=[", dpid, dpty, dplen);
    PR_DEBUG_HEX_ARRAY(dpvalue, dplen);
    PR_DEBUG_RAW("]\n");
    switch(dpid){
        case 0x01://onoff
            app_led_onoff(dpvalue[0]);
            break;
        case 0x02://mode:white,color,scene,music
            break;
        default:
            break;
    }
}
```

5.3 beacon 发送函数

函数	u8 frame_send(u8 head, u16 dst, u8 subcmd, u8* payload, u8 *key, u8 ttl); // 高级封装包发送	
作用	Beacon Mesh 协议栈最底层发包函数，直接将 payload 数据发送到目标地址，并且可以设置最大为 3 的 ttl。 注：应用 DEMO 中有单独介绍如何用该最底层发包函数，封装出发送不同类型别 dp、发送心跳等更应用的函数 注：再次调用会直接打断上一次的发包，进行全新一次的发包	
入参	u8 head	数据包 head
	u16 dst	目的地址
	u8 subcmd	数据包子命令
	u8* payload	数据包数据字段
	u8 *key	数据包密钥
	u8 ttl	跳数
返回		

示例：

dp 上报函数：

```
u8 payload[16];
memset(payload, 0, 16);
payload[0] = 1; // dp id = 1
payload[1] = 0x11; // dp kind = 1; dp len = 1 (各占 4bits)
payload[2] = led_onoff; // dp value
frame_send(0x08, 0x8000, 0x0A, payload, beacon_dev.beaconkey, 3);
```

5.4 beacon loop 函数

函数	u8 ty_beacon2_node_run(void)	
作用	需要被高频周期性调用，来驱动 beacon mesh 协议栈	
入参		
返回		

示例：

```
VOID_T tal_framework_loop(VOID_T){
    ty_beacon2_node_run();//beacon mesh 节点主逻辑维护进程
}
```

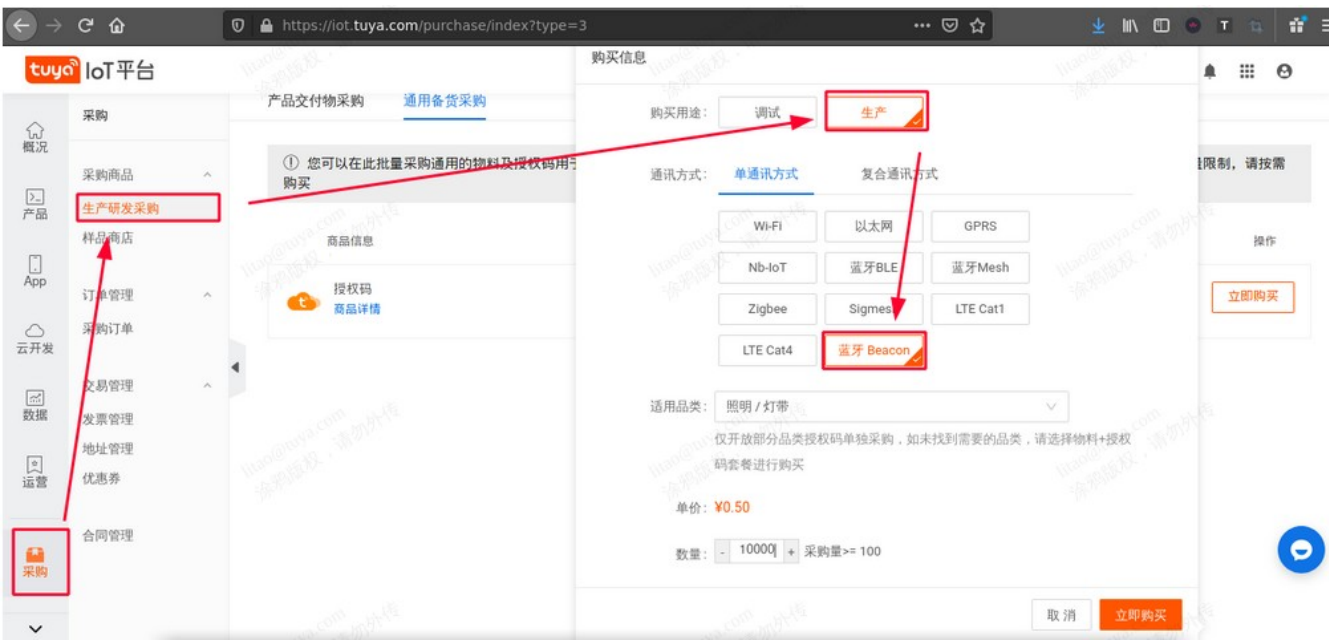
5.5 beacon 启动配网函数

函数	u8 ty_beacon2_node_pair_start(u32 timeout_us)	
作用	一旦调用，整个系统进入待配网状态	
入参	u32 timeout_us	设置配网超时时间
返回		

6、量产指导

6.1 购买授权码

当开发的产品完成功能测试、稳定性测试、老化测试、拉距测试后，对产品进行发布，然后可在涂鸦 IoT 平台采购授权码：（选择授权码清单）

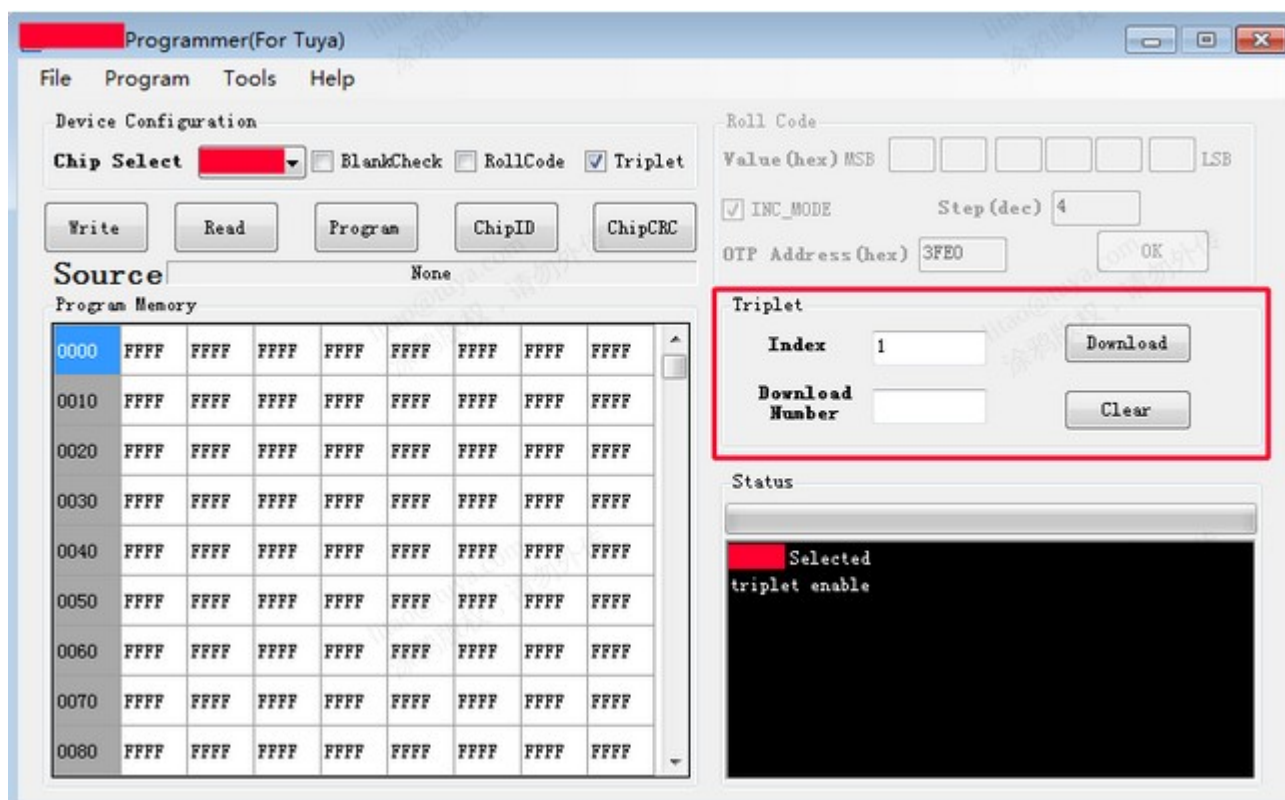


订单成功后，在邮箱中会收到授权码清单（一个表格，将表格另存为 xlsx 格式，表格中内容格式如下：）

uuid	key	mac	
tuyae15aa4acb29dc46d	RAjczGyAK4lvmPIIXGpKIKwHiyRayKsg	DC234D28EBEB	举例，不是正确的数据

6.2 芯片烧录授权

参考《SDK 开发包根目录/hardware/AK803AE-M3/chip_manual/AK801 芯片烧录器使用说明.pdf》，使用绿色免安装软件《SDK 开发包根目录/hardware/AK803AE-M3/chip_manual/烧录授权 V1.0.rar》，采用 Triplet 三元组表格授权模式，对烧写器进行写入 bin 文件和 n 条三元组信息：



- 1) Chip Select 选择 AK801，模式选择 Triplet
- 2) 烧写器连接好 AK80X Programmer(For Tuya) 后，点击 Triplet 面板中的 Clear 按钮，清空烧写器中原有授权三元组残留（擦除需要一些时间）
- 3) File -> Import Bin (选择待烧录固件) -> Write 实现待烧录固件写入离线烧写器
- 4) 在 Triplet 面板，Index 填写 1，Download Number 填写需要写入到离线烧写器中的三元组数量，然后点击 Downlod 按钮，选择 6.2 节申请的授权码清单（xlsx），实现将授权码清单写入离线烧写器
- 5) 给离线烧写器重新上电，将待烧写芯片放入卡槽中，按烧写器中间的按钮，如果烧录成功，烧录指示灯会变绿

注：代码里 authkey 和 mac 不能直接写死，而是读取对应 flash/OTP 中的内容，对其进行初始化。量产固件一定要将 BOARD_FPGA 和 PRINT_DEBUG 模式关闭，使能看门狗和 EEPROM !!!

7、其它

7.1 涂鸦平台芯片烧录授权方式介绍

- 采用基于串口通信的涂鸦标准产测授权协议，实现将授权信息写入芯片
- 直接授权码清单的方式，客户需要自己制作上位机，将 MAC\AUTHKEY\UUID 等信息直接写入芯片（AK801 就是采用这种方式，这种方式如果客户直接购买授权码清单进行大批量生产，需要经过涂鸦内部审核）

7.2 Beacon MESH 协议栈芯片移植指导

目前针对如下平台已经做好移植：

芯片	厂家	指令集类别	IDE	状态
AK801	兆焯微	平头哥	CSKY	Tuya Wind IDE
nrf52832	Nordic	32 位 ARM	KEIL	待上架

如果没有所选芯片的 Demo Project：

STEP 1. 您可以参考 AK801 SDK 中的 vendor/ak80x_adv/tuyaos 编写贵司自己平台的 tkl_bluetooth.c 文件（必须），以及相应的 tkl_gpio.c、tkl_flash.c 等外设文件（非必须）。

1) 其中 tkl_bluetooth.c 必须实现：

- OPERATE_RET tkl_ble_stack_init(UCHAR_T role)
 - 蓝牙协议栈初始化函数
- OPERATE_RET tkl_ble_gap_callback_register(CONST TKL_BLE_GAP_EVT_FUNC_CB gap_evt)
 - 蓝牙 GAP 层消息回调函数注册函数
- OPERATE_RET tkl_ble_gap_addr_set(TKL_BLE_GAP_ADDR_T CONST *p_peer_addr)
 - 蓝牙底层发广播包 MAC 地址设置函数
- OPERATE_RET tkl_ble_gap_adv_start(TKL_BLE_GAP_ADV_PARAMS_T CONST *p_adv_params)
 - 开启蓝牙广播函数
- OPERATE_RET tkl_ble_gap_adv_stop(VOID)

- 停止蓝牙广播函数
- OPERATE_RET tk1_ble_gap_adv_rsp_data_set(TKL_BLE_DATA_T CONST *p_adv, TKL_BLE_DATA_T CONST *p_scan_rsp)
 - 蓝牙广播函数内容更新函数
- OPERATE_RET tk1_ble_gap_scan_start(TKL_BLE_GAP_SCAN_PARAMS_T CONST *p_scan_params)
 - 蓝牙广播扫描函数
- OPERATE_RET tk1_ble_gap_scan_stop(VOID)
 - 蓝牙扫描停止函数

注意：开始扫描和结束扫描函数、开始广播和结束广播函数务必做到调用之后立即开始或结束，否则可能影响 beacon mesh 性能。

2) 其中非易失存储驱动要求如下：

hal_storage_256.h

```

/*****
  > File Name: hal_storage_256.h
  > Author:
  > Mail:
  > Created Time: Tue 26 Jan 2021 12:26:46 CST
  > 256 bytes fast storage(eeprom or flash ...)
  *****/

#ifndef _HAL_STORAGE_256_H
#define _HAL_STORAGE_256_H

#include "hal_sys.h"

u8 hal_storage_256_init(void);
u8 hal_storage_256_read_bytes(u16 offset, u8* pdata, u8 len); //offset 0~0xFF
u8 hal_storage_256_write_bytes(u16 offset, u8* pdata, u8 len);

#endif

```

协议栈不会高频调用读写，但是建议实现读写时采用增量式双备份方式。

3) 其中 us 级别的系统滴答驱动要求如下:

hal_clock.h

```

/*****
> File Name: hal_clock.h
> Author:
> Mail:
> Created Time: Wed 03 Jul 2019 22:09:57 CST
*****/

#ifndef _HAL_CLOCK_H
#define _HAL_CLOCK_H

#include "hal_sys.h"

#define HAL_CLOCK_HZ 1000000 //tick 增加的频率（某些系统直接等于系统主频）

enum{//对 1S\1MS\1US 等于多少 tick 的定义
//how much 1us = ? ticks
    HAL_CLOCK_1S_TICKS = HAL_CLOCK_HZ,
    HAL_CLOCK_1MS_TICKS = (HAL_CLOCK_1S_TICKS / 1000),
    HAL_CLOCK_1US_TICKS = (HAL_CLOCK_1MS_TICKS / 1000),
};

extern u32 hal_clock_get_system_tick(void);//获取当前 tick
extern u32 hal_clock_time_exceed(u32 ref, u32 span_us);//ref 填写 old_tick，该函数能够计算当前 tick-old_tick 是否超过了设定时间 span_us，超过返回 true（非阻塞）

#endif

```

STEP 2. 同时将所用平台的最小可运行 DEMO + 该 DEMO 的编译环境的搭建指导发给涂鸦（DB 或 TuyaIoTOS 论坛），涂鸦内部会根据该平台的编译链情况，生成该平台对应的 beacon mesh 协议栈库给到您。

STEP 3. 最终将下面适配好的文件全部加入工程：

适配文件名	作用
hal_storage_256.h	存储
hal_storage_256.c	
hal_clock.h	定时
hal_clock.c	
hal_sys.h	原厂 SDK 常用系统头文件汇总
hal_types.h	重定义 u8\u16\u32\s8\s16\s32
board.h	hal 层头文件汇总

libtal_beacon2.a libtal_xxtea.a include/*.h	涂鸦给的基于目标平台的 Beacon Mesh 协议栈库
---	------------------------------

然后参考上述 AK801 简单 Beacon Mesh 应用 DEMO，编写目标平台最小 DEMO，并进行调试。

注意：上述接口文件适配好之后，最好先自测下各个函数的功能是否能达到要求，然后在进行集成测试 Beacon Mesh 协议栈（因为协议栈本身高度集成且没有 LOG，不方便 DEBUG）

8、FAQ

有问题，可以到涂鸦开发者论坛交流：

<https://www.tuyaos.com/>